Exploratory Queries

1) **What are the highest and lowest transaction amounts?**

```
SELECT MAX (total_spent) AS highest_sale, MIN (total_spent) AS lowest_sale
FROM cafe_sales;
```

2) **Find menu items that contain the word 'Coffee'**

```
SELECT transaction_id,item,total_spent
FROM cafe_sales
WHERE Item LIKE ''%Coffee%';
```

3) **What is the revenue when comparing in-store and takeaway orders?**

```
SELECT location, COUNT (*) AS Orders, SUM (total_spent) as REVENUE, AVG (total_spent)
AS avg_order_value
FROM cafe_sales
GROUP BY location;
```

4) **What are high value transactions?**

```
SELECT* FROM cafe_sales
WHERE total_spent >15
```

5) **How many transactions were made for each payment method?**

```
SELECT payment_method, COUNT (*) AS transaction_count
FROM cafe_sales
GROUP BY payment_method
ORDER BY transaction_count DESC;
```

Business Queries

1) **Which items perform best in store versus takeaway?**

```
SELECT item,location, SUM (total_spent) AS revenue,COUNT(*) as ORDERS
FROM cafe_sales
GROUP BY item,location
ORDER BY revenue DESC;
```

2) **What is the total revenue and average spent?**

```
SELECT SUM (total_spent) AS total_revenue
FROM cafe_sales;
```

```
SELECT AVG(total_spent) AS avg_spend
FROM cafe_sales;
```

3) **Which menu items consistently perform well across various months?**

```
SELECT
  strftime('%Y-%m', transaction_date) AS sales_month,
  item,
  COUNT(transaction_ID) AS transactions,
  SUM(quantity) AS units_sold,
  SUM(total_spent) AS total_revenue
FROM cafe_sales
WHERE item IS NOT NULL
  AND item NOT IN ('ERROR', 'UNKNOWN')
  AND transaction_date IS NOT NULL
  AND transaction_date != ''
  AND total_spent IS NOT NULL
  AND total_spent NOT IN ('ERROR', 'UNKNOWN')
  AND quantity IS NOT NULL
  AND quantity NOT IN ('ERROR', 'UNKNOWN')
GROUP BY
  strftime('%Y-%m', transaction_date),
  item
ORDER BY sales_month, total_revenue DESC;
```

4) **Is the cafe's revenue increasing over time and how does the daily revenue compare with the recent average?**

```
WITH daily_sales AS (
  SELECT
    transaction_date,
    SUM(CAST(total_spent AS REAL)) AS daily_revenue
  FROM cafe_sales
  WHERE transaction_date IS NOT NULL
    AND transaction_date != ''
    AND total_spent IS NOT NULL
    AND total_spent NOT IN ('ERROR', 'UNKNOWN')
  GROUP BY transaction_date
)
SELECT
  transaction_date,
  daily_revenue,
  SUM(daily_revenue) OVER (
    ORDER BY transaction_date
    ROWS BETWEEN UNBOUNDED PRECEDING
    AND CURRENT ROW
  ) AS running_total_revenue,
  ROUND(
    AVG(daily_revenue) OVER (
      ORDER BY transaction_date
      ROWS BETWEEN 6 PRECEDING
      AND CURRENT ROW
    ), 2
  ) AS seven_day_moving_average
FROM daily_sales
ORDER BY transaction_date;
```